

Text Classification and Naive Bayes

The Task of Text Classification

Is this spam?

Subject: Important notice!

From: Stanford University <newsforum@stanford.edu>

Date: October 28, 2011 12:34:16 PM PDT

To: undisclosed-recipients;;

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

<http://www.123contactform.com/contact-form-StanfordNew1-236335.html>

Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information about the new services.

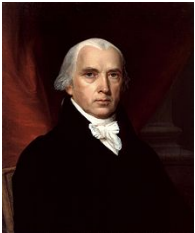
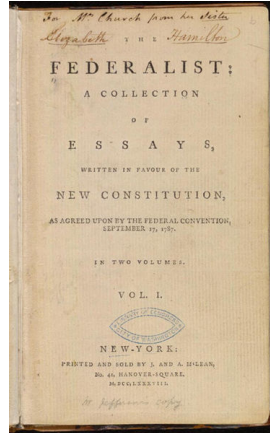
© Stanford University. All Rights Reserved.

Who wrote which Federalist papers?

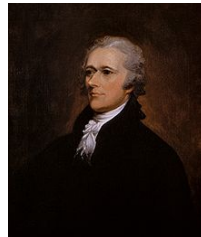
1787-8: anonymous essays try to convince New York to ratify U.S Constitution: Jay, Madison, Hamilton.

Authorship of 12 of the letters in dispute

1963: solved by Mosteller and Wallace using Bayesian methods



James Madison



Alexander Hamilton

What is the subject of this medical article?

MEDLINE Article



MeSH Subject Category Hierarchy

Antagonists and Inhibitors

Blood Supply

Chemistry

Drug Therapy

Embryology

Epidemiology

...

Positive or negative review?

- + *...zany characters and richly applied satire, and some great plot twists*
- *It was pathetic. The worst part about it was the boxing scenes...*
- + *...awesome caramel sauce and sweet toasty almonds. I love this place!*
- *...awful pizza and ridiculously overpriced...*

Positive or negative movie review?

+ ...zany characters and **richly** applied satire, and some **great** plot twists

— It was **pathetic**. The **worst** part about it was the boxing scenes...

+ ...**awesome** caramel sauce and sweet toasty almonds. I **love** this place!

— ...**awful** pizza and **ridiculously** overpriced...

Basic Sentiment Classification

Sentiment analysis is the detection of **attitudes**

Simple task we focus on in this chapter

- Is the attitude of this text positive or negative?

We return to affect classification in later chapters

Summary: Text Classification

Sentiment analysis

Spam detection

Authorship identification

Language Identification

Assigning subject categories, topics, or genres

...

Text Classification: definition

Input:

- a document d
- a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$

Output: a predicted class $c \in C$

Classification Methods: Hand-coded rules

Rules based on combinations of words or other features

- spam: black-list-address OR (“dollars” AND “you have been selected”)

Accuracy can be high

- If rules carefully refined by expert

But building and maintaining these rules is expensive

Classification Methods:

Supervised Machine Learning

Input:

- a document d
- a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$
- A training set of m hand-labeled documents $(d_1, c_1), \dots, (d_m, c_m)$

Output:

- a learned classifier $\gamma: d \rightarrow c$

Classification Methods: Supervised Machine Learning

Any kind of classifier

- Naïve Bayes
- Logistic regression
- Neural networks
- k-Nearest Neighbors
- ...

Generative Classifier

Generative classifiers (like Chat-GPT) look at patterns in data and generates a probability distribution that describes those patterns

- Given a large number of cat pictures, it will identify patterns that appear in common, such as ear shape, eye size, etc
- It then builds a probability model for what features are likely to indicate the picture is a cat
- When given an unclassified picture, it compares the features of the picture to its model of a cat and returns the likelihood that the picture is, indeed a cat.

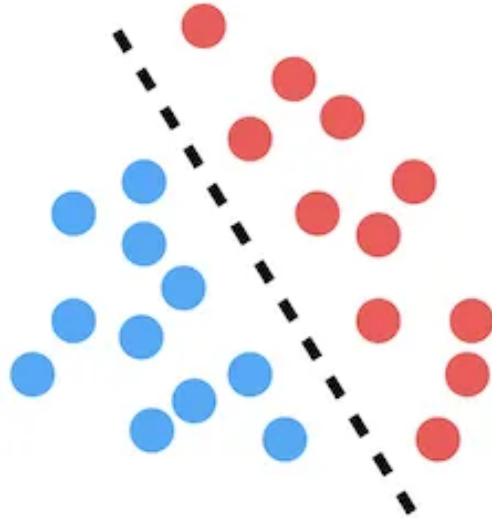
Generative Classifier

- Thus, a model is *generated* by comparing the documents of the training data and finding patterns that appear in that data
- In NLP, these patterns are usually words, N-gram, Named Entities, and other tokens

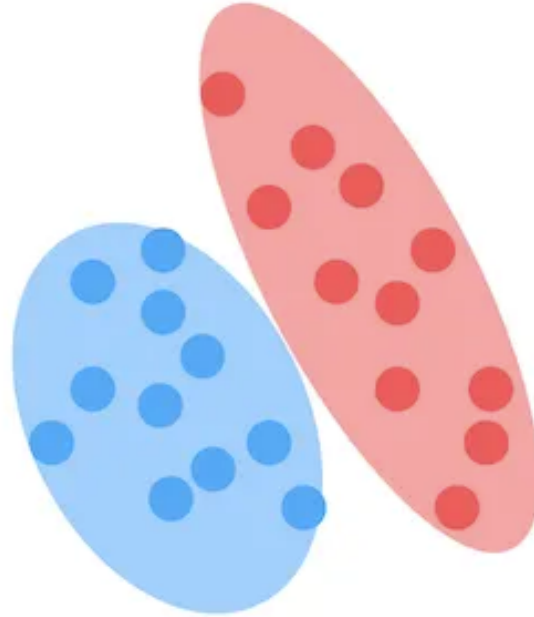
Discriminative Classifier

- A discriminative classifier doesn't try and generate the full model of the class, but instead tries to learn the decision boundary that separates the classes
- It identifies the features that are most useful in distinguishing between one class and another

Discriminative



Generative



Naive Bayes is a **Generative Classifier**
Logistic Regression is a **Discriminative Classifier**

Generative vs Discriminative

Generative		Discriminative	
Pros	Cons	Pros	Cons
Can handle complex patterns	May overfit on large datasets	High Accurac	May generalize poorly on new data
Useful for small datasets	Computationally expensive	Flexible, can be trained on many ML algorithms	Requires a lot of data
Can learn from unlabeled data	Sensitive to outliers and severe imbalance	Works well with imbalanced data	Does not handle missing data efficiently
Can capture joint distribution of input and output			

Text Classification and Naive Bayes

The Task of Text Classification

Text Classification and Naive Bayes

The Naive Bayes Classifier

Naive Bayes Intuition

Simple ("naive") classification method based on Bayes rule

Relies on very simple representation of document

- **Bag of words**

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

The bag of words representation

$Y($

seen	2
sweet	1
whimsical	1
recommend	1
happy	1
...	...

$) = C$




Bayes Rule of Inference

$$P(H | E) = \frac{P(E | H) \cdot P(H)}{P(E)}$$

H: Hypothesis

P(H): Prior Probability

E: Evidence

P(H|E): Posterier Probability

P(E | H): Likelihood

Bayes' Rule Applied to Documents and Classes

- For a document d and a class c

$$P(c|d) = \frac{P(d|c)P(c)}{P(d)}$$

Naive Bayes Classifier (I)

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(c | d)$$

MAP is “maximum a posteriori” = most likely class

$$= \operatorname{argmax}_{c \in C} \frac{P(d | c) P(c)}{P(d)}$$

Bayes Rule

$$= \operatorname{argmax}_{c \in C} P(d | c) P(c)$$

Dropping the denominator

Naive Bayes Classifier (II)

"Likelihood"

"Prior"

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(d | c) P(c)$$

$$= \operatorname{argmax}_{c \in C} P(x_1, x_2, \dots, x_n | c) P(c)$$

Document d
represented as
features x1..xn

Multinomial Naive Bayes Independence Assumptions

$$P(x_1, x_2, \dots, x_n \mid c)$$

Bag of Words assumption: Assume position doesn't matter

Conditional Independence: Assume the feature probabilities $P(x_i \mid c_j)$ are independent given the class c .

$$P(x_1, \dots, x_n \mid c) = P(x_1 \mid c) \cdot P(x_2 \mid c) \cdot P(x_3 \mid c) \cdot \dots \cdot P(x_n \mid c)$$

Multinomial Naive Bayes Classifier

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(x_1, x_2, \dots, x_n | c) P(c)$$

$$C_{NB} = \operatorname{argmax}_{c \in C} P(c_j) \prod_{x \in X} P(x | c)$$

Applying Multinomial Naive Bayes Classifiers to Text Classification

positions $\leftarrow$ all word positions in test document

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j)$$

Problems with multiplying lots of probs

There's a problem with this:

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j)$$

Multiplying lots of probabilities can result in floating-point underflow!

.0006 * .0007 * .0009 * .01 * .5 * .000008....

Idea: Use logs, because $\log(ab) = \log(a) + \log(b)$

We'll sum logs of probabilities instead of multiplying probabilities!

We actually do everything in log space

Instead of this:

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j)$$

This:

$$c_{NB} = \operatorname{argmax}_{c_j \in C} \left[\log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i | c_j) \right]$$

Notes:

1) Taking log doesn't change the ranking of classes!

The class with highest probability also has highest log probability!

2) It's a linear model:

Just a max of a sum of weights: a **linear** function of the inputs

So Naïve Bayes is a **linear classifier**

Text Classification and Naive Bayes

The Naive Bayes Classifier

Text Classification and Naïve Bayes

Naive Bayes: Learning

Learning the Multinomial Naive Bayes Model

First attempt: maximum likelihood estimates

- simply use the frequencies in the data

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}}$$

$$\hat{P}(w_i | c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)}$$

Parameter estimation

$$\hat{P}(w_i | c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)}$$

fraction of times word w_i appears
among all words in documents of topic c_j

Create mega-document for topic j by concatenating all docs in this topic

- Use frequency of w in mega-document

Problem with Maximum Likelihood

What if we have seen no training documents with the word *fantastic* and classified in the topic **positive** (*thumbs-up*)?

$$\hat{P}(\text{"fantastic"} \mid \text{positive}) = \frac{\text{count}(\text{"fantastic"}, \text{positive})}{\sum_{w \in V} \text{count}(w, \text{positive})} = 0$$

Zero probabilities cannot be conditioned away, no matter the other evidence!

$$c_{MAP} = \operatorname{argmax}_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

Laplace (add-1) smoothing for Naïve Bayes

$$\begin{aligned}\hat{P}(w_i | c) &= \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} (\text{count}(w, c) + 1)} \\ &= \frac{\text{count}(w_i, c) + 1}{\left(\sum_{w \in V} \text{count}(w, c) \right) + |V|}\end{aligned}$$

Multinomial Naïve Bayes: Learning

- From training corpus, extract *Vocabulary*

Calculate $P(c_j)$ terms

- For each c_j in C do

$docs_j \leftarrow$ all docs with class $= c_j$

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

- Calculate $P(w_k | c_j)$ terms

- $Text_j \leftarrow$ single doc containing all $docs_j$

- For each word w_k in *Vocabulary*

$n_k \leftarrow$ # of occurrences of w_k in $Text_j$

$$P(w_k | c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha | \text{Vocabulary} |}$$

Unknown words

What about unknown words

- that appear in our test data
- but not in our training data or vocabulary?

We **ignore** them

- Remove them from the test document!
- Pretend they weren't there!
- Don't include any probability for them at all!

Why don't we build an unknown word model?

- It doesn't help: knowing which class has more unknown words is not generally helpful!

Stop words

Some systems ignore stop words

- **Stop words:** very frequent words like *the* and *a*.
 - Sort the vocabulary by word frequency in training set
 - Call the top 10 or 50 words the **stopword list**.
 - Remove all stop words from both training and test sets
 - As if they were never there!

But removing stop words doesn't usually help

- So in practice most NB algorithms use **all** words and **don't** use stopwords lists

function TRAIN NAIVE BAYES(D, C) **returns** $\log P(c)$ and $\log P(w|c)$

for each class $c \in C$ # Calculate $P(c)$ terms

N_{doc} = number of documents in D

N_c = number of documents from D in class c

$\logprior[c] \leftarrow \log \frac{N_c}{N_{doc}}$

$V \leftarrow$ vocabulary of D

$bigdoc[c] \leftarrow$ **append**(d) **for** d $\in$ D **with** class c

for each word w in V # Calculate $P(w|c)$ terms

$count(w, c) \leftarrow$ # of occurrences of w in $bigdoc[c]$

$\loglikelihood[w, c] \leftarrow \log \frac{count(w, c) + 1}{\sum_{w' \text{ in } V} (count(w', c) + 1)}$

return $\logprior, \loglikelihood, V$

function TEST NAIVE BAYES($testdoc, \logprior, \loglikelihood, C, V$) **returns** best c

for each class $c \in C$

$sum[c] \leftarrow \logprior[c]$

for each position i in $testdoc$

$word \leftarrow testdoc[i]$

if $word \in V$

$sum[c] \leftarrow sum[c] + \loglikelihood[word, c]$

return $\operatorname{argmax}_c sum[c]$

Text Classification and Naive Bayes

Naive Bayes: Learning

Text Classification and Naive Bayes

Sentiment and Binary Naive Bayes

Let's do a worked sentiment example!

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Build Vocabulary

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$V = \{\text{and, boring, energy, entirely, few, film, fun, just, lacks, laughs, most, no, of, plain, powerful, predictable, summer, surprises, the, very}\} \rightarrow |V|=20$

Prior Probability

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$$P(-) = 3 \rightarrow 3/5 = 0.6$$

$$P(+) = 2 \rightarrow 2/5 = 0.4$$

Define classes

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$N(-) = \{\text{and, and, boring, energy, entirely, few, just, lacks, laughs, no, plain, predictable, surprises, very}\} \rightarrow |N-| = 14$

$N(+) = \{\text{film, fun, most, of, powerful, summer, the, the, very}\} \rightarrow |N+| = 9$

Train models with Laplace Smoothing

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$$P_{-}(\text{and}) = (C_{-}+1) / (|N_{-}| + |V|) =$$

$$(2 + 1) / (14 + 20) = 3/34 = 0.0882,$$

$$P_{-}(\text{boring}) = (1 + 1) / (14 + 20) = 2/34 = 0.0588$$

...

Run Test through the classifier

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Test={predicable, with, no , fun}

"with" does not appear in V, so
drop it ->

Test={predictable, no, fun}

Likelihood based on trained models

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$P(\text{"predictable"}|-) \rightarrow 2/34$

$P(\text{"no"}|-) \rightarrow 2/34$

$P(\text{"fun"}|-) \rightarrow 1/34$

$$P(-)P(\text{Test}|-) = 3/5 * 2/34 * 2/34 * 1/34$$
$$= 6.1 \times 10^{-5}$$

Likelihood based on trained models

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

$P(\text{"predictable"}|+) \rightarrow 1/34$

$P(\text{"no"}|+) \rightarrow 1/34$

$P(\text{"fun"}|+) \rightarrow 2/34$

$$P(+| \text{Test}) = 2/5 * 1/34 * 1/34 * 2/34$$
$$= 3.2 \times 10^{-5}$$

A worked sentiment example with add-1 smoothing

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

1. Prior from training:

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}} \quad \begin{array}{l} P(-) = 3/5 \\ P(+) = 2/5 \end{array}$$

2. Drop "with"

3. Likelihoods from training:

$$p(w_i | c) = \frac{\text{count}(w_i, c) + 1}{\left(\sum_{w \in V} \text{count}(w, c) \right) + |V|}$$

$$\begin{array}{ll} P(\text{"predictable"}|-) = \frac{0+1}{14+20} & P(\text{"predictable"}|+) = \frac{0+1}{9+20} \\ P(\text{"no"}|-) = \frac{1+1}{14+20} & P(\text{"no"}|+) = \frac{0+1}{9+20} \\ P(\text{"fun"}|-) = \frac{0+1}{14+20} & P(\text{"fun"}|+) = \frac{1+1}{9+20} \end{array}$$

4. Scoring the test set:

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

Optimizing for sentiment analysis

For tasks like sentiment, word **occurrence** seems to be more important than word **frequency**.

- The occurrence of the word *fantastic* tells us a lot
- The fact that it occurs 5 times may not tell us much more.

Binary multinominal naive bayes, or binary NB

- Clip our word counts at 1
- Note: this is different than Bernoulli naive bayes; see the textbook at the end of the chapter.

Binary multinominal naive Bayes

Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

Binary multinomial naive Bayes

Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

	NB Counts	
	+	–
and	2	0
boxing	0	1
film	1	0
great	3	1
it	0	1
no	0	1
or	0	1
part	0	1
pathetic	0	1
plot	1	1
satire	1	0
scenes	1	2
the	0	2
twists	1	1
was	0	2
worst	0	1

Binary multinominal naive Bayes

Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts	
	+	–
and	2	0
boxing	0	1
film	1	0
great	3	1
it	0	1
no	0	1
or	0	1
part	0	1
pathetic	0	1
plot	1	1
satire	1	0
scenes	1	2
the	0	2
twists	1	1
was	0	2
worst	0	1

Binary multinominal naive Bayes

Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	–	+	–
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1

Counts can still be 2! Binarization is within-doc!

Text Classification and Naive Bayes

Sentiment and Binary Naive Bayes

Text Classification and Naive Bayes

More on Sentiment Classification

Sentiment Classification: Dealing with Negation

I really like this movie

I really **don't** like this movie

Negation changes the meaning of "like" to negative.

Negation can also change negative to positive-ish

- **Don't** dismiss this film
- **Doesn't** let us get bored

Sentiment Classification: Dealing with Negation

Das, Sanjiv and Mike Chen. 2001. Yahoo! for Amazon: Extracting market sentiment from stock message boards. In Proceedings of the Asia Pacific Finance Association Annual Conference (APFA).

Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. 2002. Thumbs up? Sentiment Classification using Machine Learning Techniques. EMNLP-2002, 79—86.

Simple baseline method:

Add NOT_ to every word between negation and following punctuation:

didn't like this movie , but I



didn't NOT_like NOT_this NOT_movie but I

Sentiment Classification: Lexicons

Sometimes we don't have enough labeled training data

In that case, we can make use of pre-built word lists

Called **lexicons**

There are various publicly available lexicons

MPQA Subjectivity Cues Lexicon

Theresa Wilson, Janyce Wiebe, and Paul Hoffmann (2005). Recognizing Contextual Polarity in Phrase-Level Sentiment Analysis. Proc. of HLT-EMNLP-2005.

Riloff and Wiebe (2003). Learning extraction patterns for subjective expressions. EMNLP-2003.

Home page: https://mpqa.cs.pitt.edu/lexicons/subj_lexicon/

6885 words from 8221 lemmas, annotated for intensity (strong/weak)

- 2718 positive
- 4912 negative

+ : *admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great*

– : *awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate*

The General Inquirer

Philip J. Stone, Dexter C Dunphy, Marshall S. Smith, Daniel M. Ogilvie. 1966. The General Inquirer: A Computer Approach to Content Analysis. MIT Press

- Home page: <http://www.wjh.harvard.edu/~inquirer>
- List of Categories: <http://www.wjh.harvard.edu/~inquirer/homecat.htm>
- Spreadsheet: <http://www.wjh.harvard.edu/~inquirer/inquirerbasic.xls>

Categories:

- Positive (1915 words) and Negative (2291 words)
- Strong vs Weak, Active vs Passive, Overstated versus Understated
- Pleasure, Pain, Virtue, Vice, Motivation, Cognitive Orientation, etc

Free for Research Use

Using Lexicons in Sentiment Classification

Add a feature that gets a count whenever a word from the lexicon occurs

- E.g., a feature called "**this word occurs in the positive lexicon**" or "**this word occurs in the negative lexicon**"

Now all positive words (*good, great, beautiful, wonderful*) or negative words count for that feature.

Using 1-2 features isn't as good as using all the words.

- But when training data is sparse or not representative of the test set, dense lexicon features can help

Summary: Naive Bayes is Not So Naive

Very Fast, low storage requirements

Work well with very small amounts of training data

Robust to Irrelevant Features

Irrelevant Features cancel each other without affecting results

Very good in domains with many equally important features

Decision Trees suffer from *fragmentation* in such cases – especially if little data

Optimal if the independence assumptions hold: If assumed independence is correct, then it is the Bayes Optimal Classifier for problem

A good dependable baseline for text classification

- **But we will see other classifiers that give better accuracy**

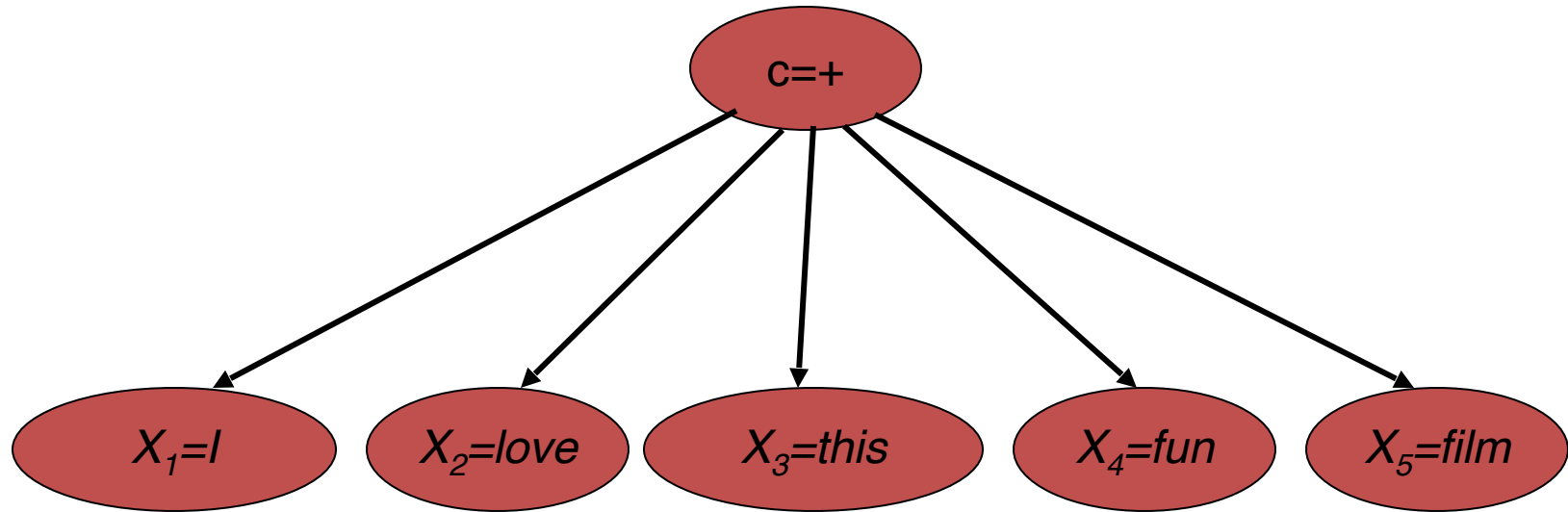
Text Classification and Naive Bayes

More on Sentiment Classification

Text Classification and Naïve Bayes

Naïve Bayes: Relationship to Language Modeling

Generative Model for Multinomial Naïve Bayes



Naïve Bayes and Language Modeling

Naïve Bayes classifiers can use any sort of feature

- URL, email address, dictionaries, network features

But if, as in the previous slides

- We use **only** word features
- we use **all** of the words in the text (not a subset)

Then

- Naïve Bayes has an important similarity to language modeling.

Each class = a unigram language model

Assigning each word: $P(\text{word} \mid c)$

Assigning each sentence: $P(s \mid c) = \prod P(\text{word} \mid c)$

Class <i>pos</i>						
0.1	I					
0.1	love	<u>I</u>	<u>love</u>	<u>this</u>	<u>fun</u>	<u>film</u>
0.01	this	0.1	0.1	.05	0.01	0.1
0.05	fun					
0.1	film					

$$P(s \mid \text{pos}) = 0.0000005$$

...

Naïve Bayes as a Language Model

Which class assigns the higher probability to s?

Model pos	
0.1	I
0.1	love
0.01	this
0.05	fun
0.1	film

Model neg	
0.2	I
0.001	love
0.01	this
0.005	fun
0.1	film

I	love	this	fun	film
0.1	0.1	0.01	0.05	0.1
0.2	0.001	0.01	0.005	0.1

$$P(s|\text{pos}) > P(s|\text{neg})$$

Text Classification and Naïve Bayes

Naïve Bayes: Relationship to Language Modeling

Text Classification and Naïve Bayes

Precision, Recall, and F
measure

Evaluation

Let's consider just binary text classification tasks

Imagine you're the CEO of Delicious Pie Company

You want to know what people are saying about your pies

So you build a "Delicious Pie" tweet detector

- Positive class: tweets about Delicious Pie Co
- Negative class: all other tweets

The 2-by-2 confusion matrix

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Evaluation: Accuracy

Why don't we use **accuracy** as our metric?

Imagine we saw 1 million tweets

- 100 of them talked about Delicious Pie Co.
- 999,900 talked about something else

We could build a dumb classifier that just labels every tweet "not about pie"

- It would get 99.99% accuracy!!! Wow!!!!
- But useless! Doesn't return the comments we are looking for!
- That's why we use **precision** and **recall** instead

Evaluation: Precision

% of items the system detected (i.e., items the system labeled as positive) that are in fact positive (according to the human gold labels)

$$\textit{Precision} = \frac{\textit{TruePositives}}{\textit{TruePositives} + \textit{FalsePositives}}$$

Evaluation: Recall

% of items actually present in the input that were correctly identified by the system.

$$\text{Recall} = \frac{\text{TruePositives}}{\text{TruePositives} + \text{FalseNegatives}}$$

Why Precision and recall

Our dumb pie-classifier

- Just label nothing as "about pie"

Accuracy=99.99%

but

Recall = 0

- (it doesn't get any of the 100 Pie tweets)

Precision and recall, unlike accuracy, emphasize true positives:

- finding the things that we are supposed to be looking for.

A combined measure: F

F measure: a single number that combines P and R:

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

We almost always use balanced F_1 (i.e., $\beta = 1$)

$$F_1 = \frac{2PR}{P + R}$$

Text Classification and Naive Bayes

Evaluation with more than
two classes

Confusion Matrix for 3-class classification

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	precision_u = $\frac{8}{8+10+1}$
	normal	5	60	50	precision_n = $\frac{60}{5+60+50}$
	spam	3	30	200	precision_s = $\frac{200}{3+30+200}$
		recall_u = $\frac{8}{8+5+3}$	recall_n = $\frac{60}{10+60+30}$	recall_s = $\frac{200}{1+50+200}$	

How to combine P/R from 3 classes to get one metric

Macroaveraging:

- compute the performance for each class, and then average over classes

Microaveraging:

- collect decisions for all classes into one confusion matrix
- compute precision and recall from that table.

Macroaveraging and Microaveraging

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = .73$$

$$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = .60$$

Text Classification and Naive Bayes

Test Sets and Cross-Validation

Development Test Sets ("Devsets") and Cross-validation

Training set

Development Test Set

Test Set

Train on training set, tune on devset, report on testset

- This avoids overfitting ('tuning to the test set')
- More conservative estimate of performance
- But paradox: want as much data as possible for training, and as much for dev; how to split?

Cross-validation: multiple splits

Pool results over splits, Compute pooled dev performance



Text
Classification
and Naive
Bayes

Avoiding Harms in
Classification

Harms in sentiment classifiers

Kiritchenko and Mohammad (2018) found that most sentiment classifiers assign lower sentiment and more negative emotion to sentences with African American names in them.

This perpetuates negative stereotypes that associate African Americans with negative emotions

<https://aclanthology.org/S18-2005.pdf>

Harms in toxicity classification

Toxicity detection is the task of detecting hate speech, abuse, harassment, or other kinds of toxic language

But some toxicity classifiers incorrectly flag as being toxic sentences that are non-toxic but simply mention identities like blind people, women, or gay people.

This could lead to censorship of discussion about these groups.

What causes these harms?

Can be caused by:

- Problems in the training data; machine learning systems are known to amplify the biases in their training data.
- Problems in the human labels
- Problems in the resources used (like lexicons)
- Problems in model architecture (like what the model is trained to optimize)

Mitigation of these harms is an open research area

Meanwhile: **model cards**

Model Cards

(Mitchell et al., 2019)

For each algorithm you release, document:

- training algorithms and parameters
- training data sources, motivation, and preprocessing
- evaluation data sources, motivation, and preprocessing
- intended use and users
- model performance across different demographic or other groups and environmental situations

<https://huggingface.co/blog/model-cards>

Evaluating Language Model Bias

(Mitchell et al., 2019)

For each algorithm you release, document:

- training algorithms and parameters
- training data sources, motivation, and preprocessing
- evaluation data sources, motivation, and preprocessing
- intended use and users
- model performance across different demographic or other groups and environmental situations

<https://huggingface.co/blog/evaluating-llm-bias>

Text
Classification
and Naive
Bayes

Avoiding Harms in
Classification